

Capture the Flag 7: Stop Putting Everything on One Screen

Goal: Split your Student Directory into multiple screens using Flutter navigation, while using a modal for editing student information.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

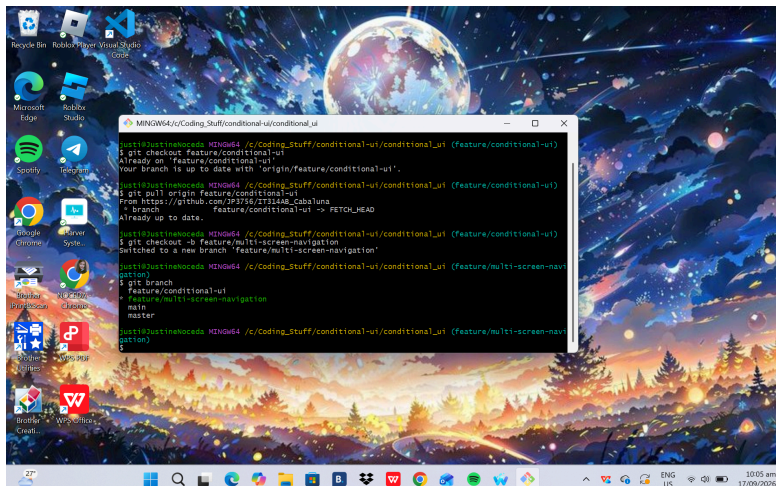
Objective: Create a new feature branch before adding multiple screens.

Create:

`feature/multi-screen-navigation`

Screenshot:

- Terminal showing your current branch



🚩 Flag 1 — Create the Student List Screen

Objective: Move your Student Directory into its own screen.

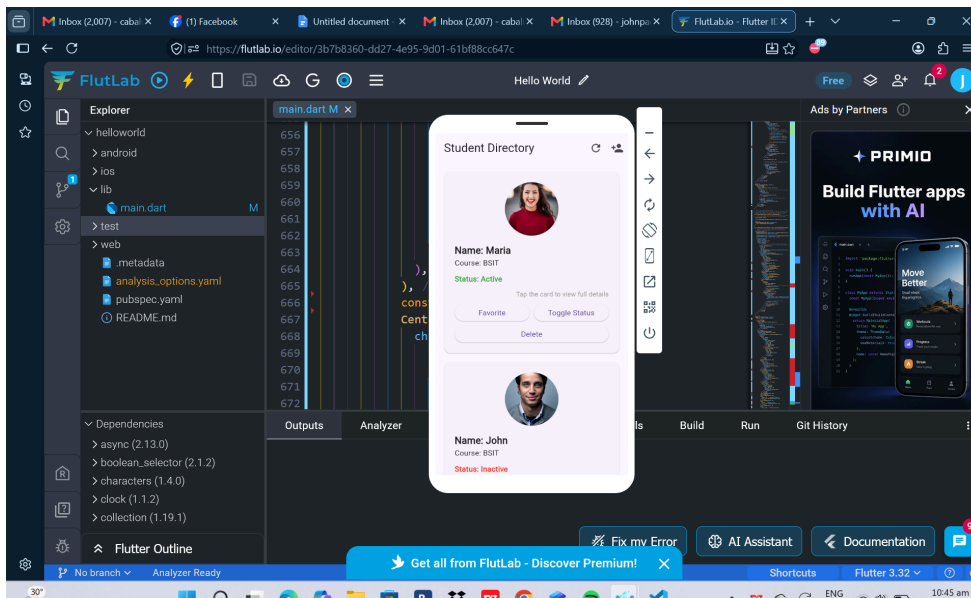
Create a dedicated **Student List Screen** containing your existing student list.

Research how Flutter creates separate screens using **Widget** classes.

Test

The Student Directory should function as its own screen.

Screenshot:



- Student List Screen

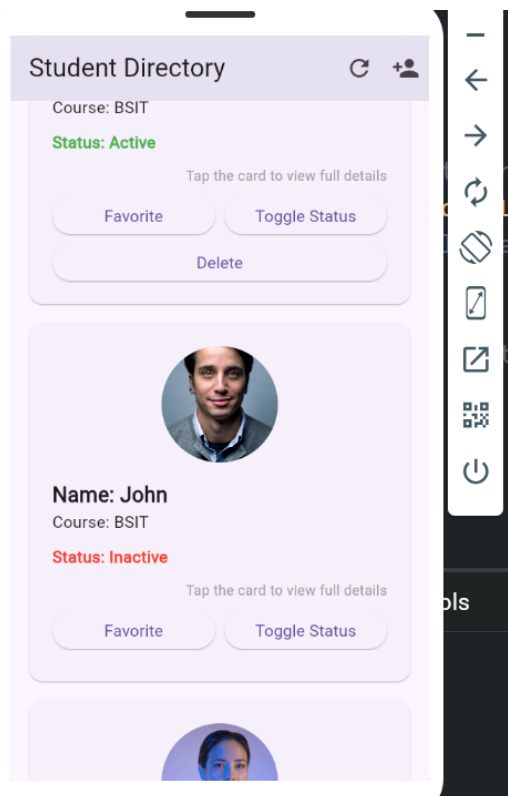
```
class StudentListScreen extends StatefulWidget {  
  const StudentListScreen({super.key});  
  
  @override  
  State<StudentListScreen> createState() {  
    return StudentListScreenState();  
  }  
}
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Student Directory'),
      actions: [
        IconButton(
          onPressed: () {
            loadStudents();
          },
          icon: const Icon(Icons.refresh),
        ),
        IconButton(
          onPressed: openAddStudent,
          icon: const Icon(Icons.person_add),
        ),
      ],
    ),
    body: showBody(),
  );
}

```

- Student List screen code



- Application showing the screen

🚩 Flag 2 — Create the Student Details Screen

Objective: Create a separate screen for viewing a student's information.

When a student is selected, navigate:

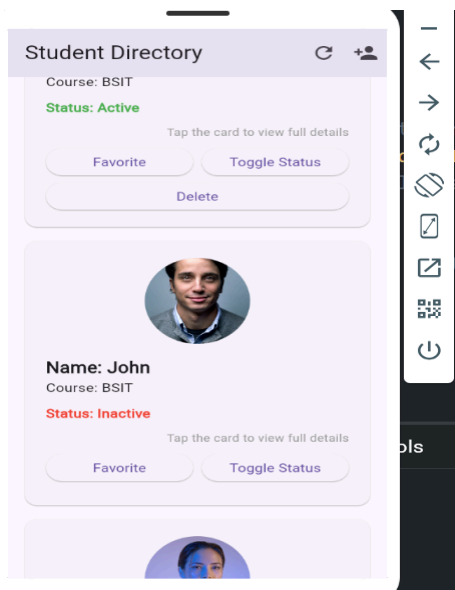
Student List → Student Details

Research Flutter's **Navigator** and how to move between screens.

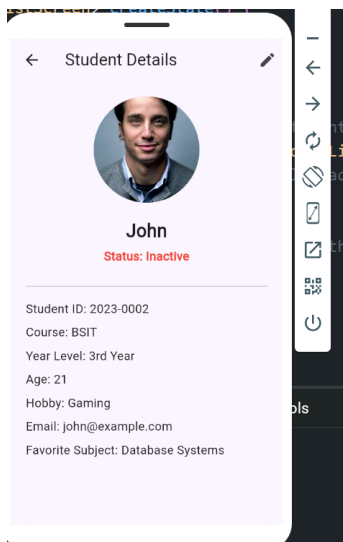
Test

1. Select a student.
2. Open the Student Details Screen. (No data displayed yet)
3. Return to the Student List using back navigation.

Screenshot:



- Student List Screen



- Student Details Screen

```
void openStudentDetails(Profile tappedStudent) async {  
  await Navigator.pushNamed(  
    context,  
    '/student-details',  
    arguments: tappedStudent,  
  );  
  
  setState(() {  
  });  
}
```

- Navigation code

🚩 Flag 3 — Pass Student Data

Objective: Pass the selected student's data to the Student Details Screen.

The Details Screen should display the correct information for the selected student.

Display relevant information such as:

- Name
- Student ID
- Course
- Status
- Other existing student data

Research how data can be passed between Flutter screens.

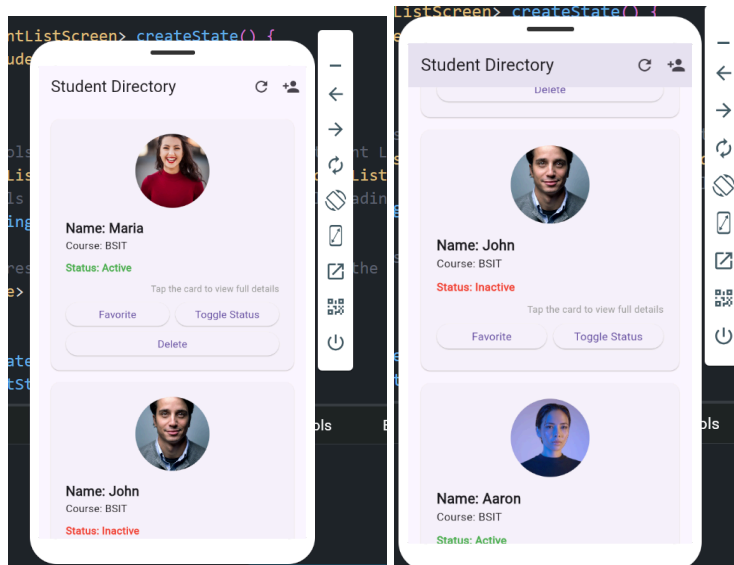
Test

Selecting different students should display their corresponding information.

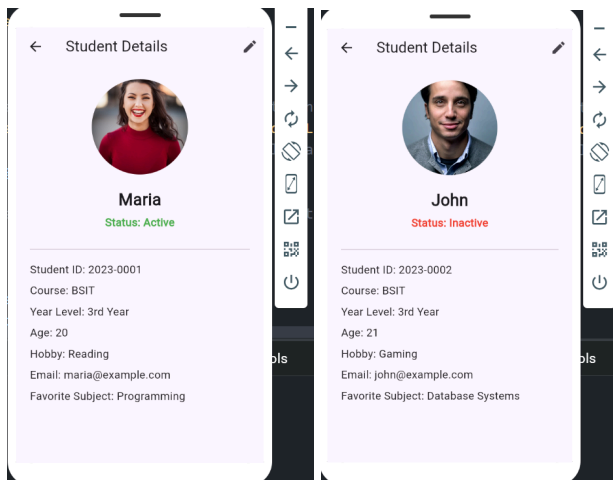
Student A → Student A's Details

Student B → Student B's Details

Screenshot:



- Selected students



- Student Details

```
onGenerateRoute: (settings) {
  if (settings.name == '/student-details') {
    final selectedStudent = settings.arguments as Profile;

    return MaterialPageRoute(
      builder: (context) {
        return StudentDetailsScreen(student: selectedStudent);
      },
    );
  }

  return null;
},
```

- Data-passing code

🚩 Flag 4 — Edit with a Modal

Objective: Allow administrators to edit student information without creating another screen.

Add an **Edit** action to the Student Details Screen.

When selected, display an editing interface using a **modal**, such as a dialog or bottom sheet.

Research Flutter's:

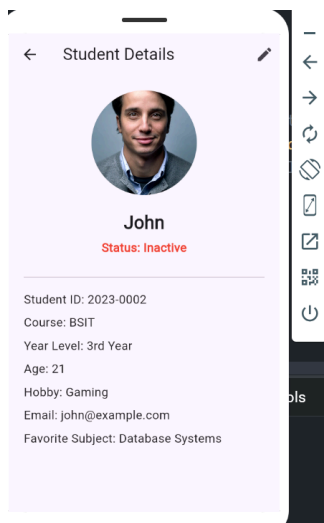
- `showDialog`
- `showModalBottomSheet`

You may choose whichever approach is appropriate for your design.

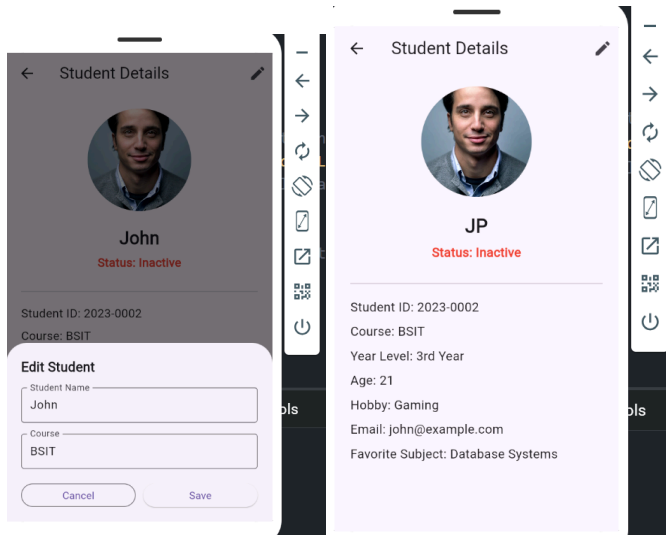
Test

1. Open Student Details.
2. Select Edit.
3. An editing modal should appear.
4. The modal should display the appropriate student's information.
5. Close or submit the modal.

Screenshot:



- Student Details Screen



- Edit modal

```
void openEditModal() {
  TextEditingController nameController = TextEditingController();
  nameController.text = widget.student.name;

  TextEditingController courseController = TextEditingController();
  courseController.text = widget.student.course;

  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    builder: (context) {
      return Padding(
        padding: EdgeInsets.only(
          left: 20,
          right: 20,
          top: 20,
          bottom: MediaQuery.of(context).viewInsets.bottom + 20,
        ),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Edit Student',
              style: TextStyle(
                fontSize: 18,

```

```
fontWeight: FontWeight.bold,
),
),
const SizedBox(height: 15),
TextField(
  controller: nameController,
  decoration: const InputDecoration(
    labelText: 'Student Name',
    border: OutlineInputBorder(),
  ),
),
const SizedBox(height: 15),
TextField(
  controller: courseController,
  decoration: const InputDecoration(
    labelText: 'Course',
    border: OutlineInputBorder(),
  ),
),
const SizedBox(height: 20),
Row(
  children: [
    Expanded(
      child: OutlinedButton(
        onPressed: () {
```

```

        Navigator.pop(context);
      },
      child: const Text('Cancel'),
    ),
  ),
  const SizedBox(width: 10),
  Expanded(
    child: ElevatedButton(
      onPressed: () {
        setState(() {
          widget.student.name = nameController.text;
          widget.student.course = courseController.text;
        });

        Navigator.pop(context);
      },
      child: const Text('Save'),
    ),
  ),
],
),
],
),
);
},
);
}

```

- Modal code

🚩 Flag 5 — Create the Add Student Screen

Objective: Create a separate screen for adding a new student.

Add an action from the Student List that navigates to:

Student List → Add Student

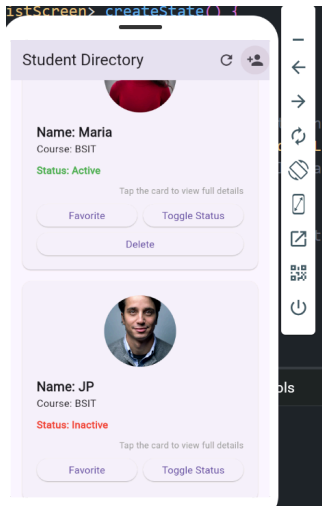
Create the **Add Student Screen**.

For now, focus on creating the screen and navigation structure.

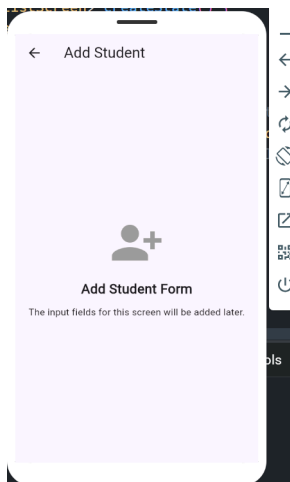
Test

Selecting Add Student should open the Add Student Screen.

Screenshot:



- Student List with Add action



- Add Student Screen

```
void openAddStudent() {
    Navigator.pushNamed(context, '/add-student');
}
```

```
IconButton(
  onPressed: openAddStudent,
  icon: const Icon(Icons.person_add),
),
```

- Navigation code

🚩 Flag 6 — Prepare the Add Student Screen

Objective: Prepare the Add Student Screen for future functionality.

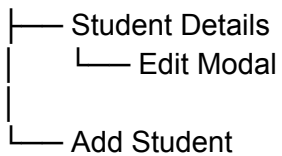
Organize the screen so it is ready to receive student input.

Do not implement the student input fields yet.

The screen should simply exist as a separate destination.

Your application should now have:

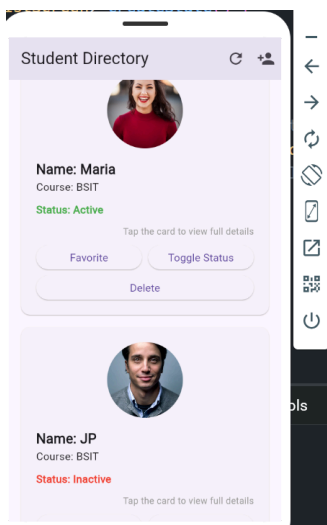
Student List



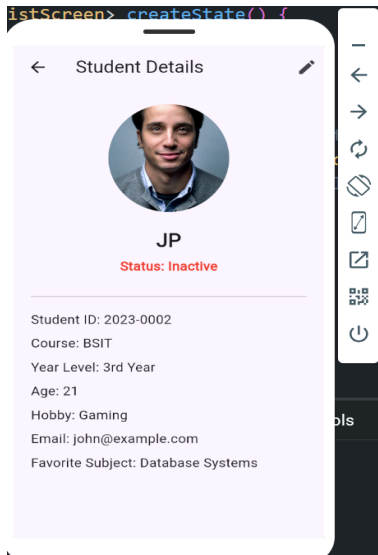
Test

You should be able to navigate between the appropriate screens without placing everything on one screen.

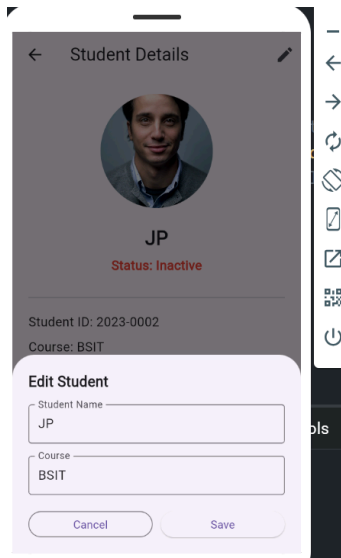
Screenshot:



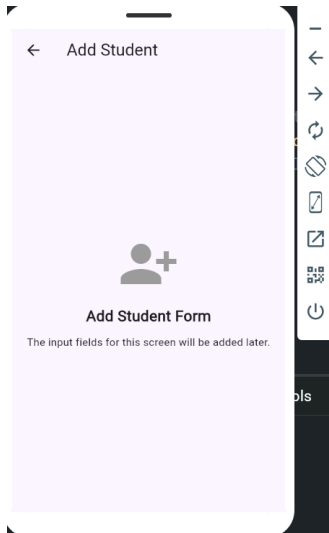
- Student List



- Student Details



- Edit Modal



- Add Student Screen

🚩 Flag 7 — Connect the Routes

Objective: Organize the application's screens using Flutter routes.

Create routes for the major screens:

/student-list
/student-details
/add-student

Research:

- Named routes
- `Navigator.pushNamed`
- Passing arguments through routes

The **Edit Modal** should not be a route.

Test

Verify that:

- Student List opens correctly.
- Student Details opens correctly.
- The correct student data is passed.
- Add Student opens correctly.
- Edit opens as a modal from Student Details.

Screenshot:

```

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Directory',

      initialRoute: '/student-list',

      routes: {
        '/student-list': (context) => const StudentListScreen(),
        '/add-student': (context) => const AddStudentScreen(),
      },

      onGenerateRoute: (settings) {
        if (settings.name == '/student-details') {
          final selectedStudent = settings.arguments as Profile;

          return MaterialPageRoute(
            builder: (context) {
              return StudentDetailsScreen(student: selectedStudent);
            },
          );
        }
      }
    );
  }
}

```

- Route configuration

```

void openStudentDetails(Profile tappedStudent) async {
  await Navigator.pushNamed(
    context,
    '/student-details',
    arguments: tappedStudent,
  );

  setState(() {
  });
}

```

```

void openAddStudent() {
  Navigator.pushNamed(context, '/add-student');
}

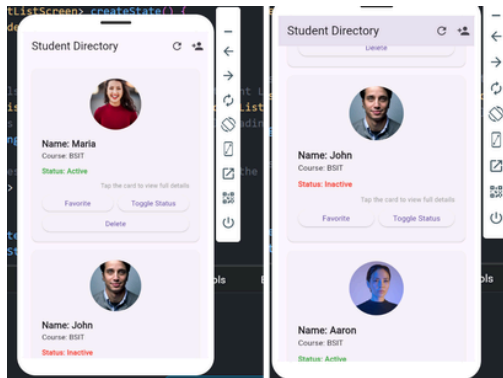
```

```

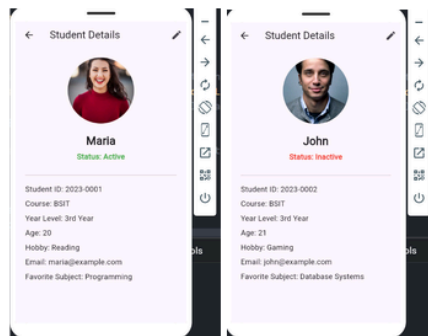
IconButton(
  onPressed: openAddStudent,
  icon: const Icon(Icons.person_add),
),

```

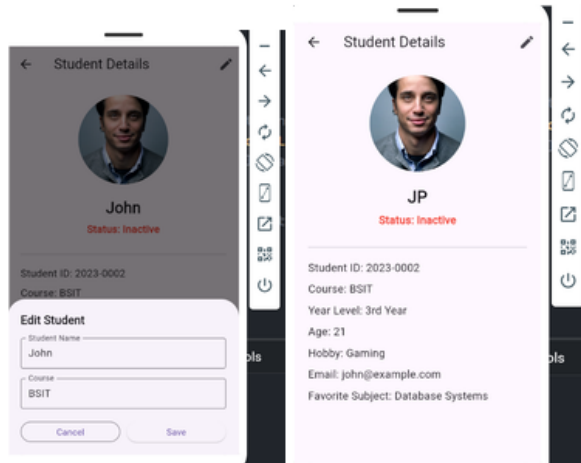
- Navigation using routes
- Student Details receiving student data



- Selected students



- Edit modal



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Multi-Screen Student Directory.

Use a proper Git commit message.

Branch:

feature/multi-screen-navigation

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit